

Chapitre 1

Erreurs d'arrondi

Notes rédigées par Raphaël JONTEF
Avec l'aide de Github Copilot

Cours enseigné par Marc Durufle

Enseirb-Matmeca
filière informatique

30 janvier 2026

Table des matières

I	Représentation des nombres en machine	2
I.1	Entiers	2
I.2	Représentation des réels	2
II	Erreurs d'arrondi	3
II.1	Opérations élémentaires	3
II.1.1	Mesure de l'erreur d'arrondi	3
II.1.2	Somme de deux réels	4
II.1.3	Produit de deux réels	5

I Représentation des nombres en machine

I.1 Entiers

Les Entiers sont codés en binaire. On écrit alors pour $n \in \mathbb{N}$:

$$n = \sum_{k=0}^{n-1} a_k 2^k$$

où $a_k \in \{0, 1\}$ est appelé *bit*.

On peut représenter les entiers signés en utilisant le bit de poids fort (le bit le plus à gauche) pour représenter le signe : 0 pour positif, 1 pour négatif.

On peut aussi utiliser la représentation en complément à 2 pour représenter les entiers signés. Dans cette représentation, un entier négatif $-n$ est représenté par $2^m - n$ où m est le nombre de bits utilisés pour représenter l'entier.

Avec une telle implementation, on peut représenter les entiers de -2^{m-1} à $2^{m-1} - 1$ et la somme, la différence, le produit, la division de deux entiers est toujours correcte modulo 2^m .

I.2 Représentation des réels

Pour $x \in \mathbb{R}^*$, on écrit :

$$x = y2^e \quad \text{avec} \quad \begin{cases} 1 \leq y < 2 \\ e \in \mathbb{Z} \end{cases}$$

On écrit alors y en binaire :

$$y = \sum_{k=0}^{+\infty} y_k 2^{-k}$$

où $y_k \in \{0, 1\}$.

On appelle *mantisse* la partie y et *exposant* la partie e .

En machine, on ne peut pas représenter un réel avec une précision infinie. On choisit donc de ne garder que t bits pour la mantisse et L bits pour l'exposant.

On note m le nombre de bits total :

$$m = 1 + t + L$$

où 1 bit est utilisé pour le signe.

On note $\mathcal{F}$ l'ensemble des réels représentables en machine avec cette précision. On a alors :

$$\mathcal{F} = \{0\} \cup \left\{ \pm \left(\sum_{k=0}^{t-1} y_k 2^{-k} \right) 2^e \mid a_0 = 1, y_k \in \{0, 1\}, e \in [e_{\min}, e_{\max}] \right\}$$

où $e_{\min} = 1 - 2^{L-1}$ et $e_{\max} = 2^{L-1} - 2$.

Définition 1.1 - *nombre dénormalisé*

Un nombre est dit dénormalisé lorsque son exposant dans la norme **IEEE754** est nul, et sa mantisse est non nulle. Dans ce cas, la mantisse est codée sans le bit implicite a_0 : $a_0 = 0$.

On appelle *overflow* le fait qu'un calcul donne un résultat supérieur à e_{max} . On appelle *underflow* le fait qu'un calcul donne un résultat inférieur à e_{min} . Dans ces deux cas, le résultat est arrondi à 0.

Exemple 1.2 - *double précision en C*

En C, le type `double` utilise 64 bits pour représenter un réel. On a donc :

- 1 bit pour le signe
- 11 bits pour l'exposant
- 52 bits pour la mantisse

La précision est alors de $2^{-52} \approx 2.22 \times 10^{-16}$ (en simple précision, elle est de $2^{-23} \approx 1.19 \times 10^{-7}$).

Remarque 1.3 - *le 0*

Pour $x = 0$, on prend $y = 0$ et $e = e_{min}$. On a donc une représentation unique du 0 en machine, au signe près.

Remarque 1.4 - *les non nuls*

Pour $x \neq 0$, on a $y \geq 1$ d'où $a_0 = 1$, sauf si on veut représenter des nombres dénormalisés (inférieurs à $2^{e_{min}} = 2^{-1024}$)

II Erreurs d'arrondi

II.1 Opérations élémentaires

II.1.1 Mesure de l'erreur d'arrondi

Pour x et y deux réels et $\star$ une opération élémentaire (addition, soustraction, multiplication, division), Pour mesurer l'erreur du calcul de $x \star y$, on note :

$$e_{\star}(x, y) = \frac{|(x + \delta x) \star (y + \delta y) - x \star y|}{|x \star y|}$$

où $|\delta x| \ll |x|$ et $|\delta y| \ll |y|$. On divise par $|x \star y|$ pour avoir une mesure relative de l'erreur d'arrondi, et non absolue, c'est-à-dire dépendante du résultat de l'opération $x \star y$.

II.1.2 Somme de deux réels

Soient $x, y \in \mathbb{R}$.

$$\begin{aligned}
 e_+(x, y) &= \frac{|(x + \delta x) + (y + \delta y) - (x + y)|}{|x + y|} \\
 &= \frac{|\delta x + \delta y|}{|x + y|} \\
 &\leq \frac{|\delta x|}{|x + y|} + \frac{|\delta y|}{|x + y|} \quad (\text{inégalité triangulaire}) \\
 &= \underbrace{\frac{|\delta x|}{|x|}}_{\text{erreur initiale sur } x} \times \underbrace{\frac{|x|}{|x + y|}}_{\text{coefficient amplificateur}} + \underbrace{\frac{|\delta y|}{|y|}}_{\text{erreur initiale sur } y} \times \underbrace{\frac{|y|}{|x + y|}}_{\text{coefficient amplificateur}}
 \end{aligned}$$

Remarque 1.5 - rôle des coefficients amplificateurs

L'erreur relative sur la somme est une combinaison linéaire des erreurs relatives initiales sur x et y , pondérées par les coefficients

$$\frac{|x|}{|x + y|} \quad \text{et} \quad \frac{|y|}{|x + y|}.$$

Lorsque x et y sont de même signe et de tailles comparables, ces coefficients sont de l'ordre de 1, et l'erreur est peu amplifiée.

En revanche, lorsque x et y sont de signes opposés et que $|x + y| \ll |x|, |y|$, les coefficients amplificateurs deviennent très grands : de petites erreurs initiales sur x et y peuvent alors produire une grande erreur relative sur la somme. Ce phénomène correspond à une perte de chiffres significatifs (annulation catastrophique, voir [Wikipédia](#)).

Remarque 1.6 - erreurs initiales

Les formules des erreurs initiales $\frac{|\delta x|}{|x|}$ et $\frac{|\delta y|}{|y|}$, proviennent de l'expression :

$$\frac{|(x + \delta x) - x|}{|x|} = \frac{|\delta x|}{|x|}$$

qui mesure l'erreur entre la valeur exacte x et la valeur approchée $x + \delta x$. Là encore on divise par $|x|$ pour avoir une mesure relative de l'erreur, et non absolue, c'est-à-dire dépendante de la valeur exacte $|x|$.

II.1.3 Produit de deux réels

Soient $x, y \in \mathbb{R}$.

$$\begin{aligned}
 e_{\times}(x, y) &= \frac{|(x + \delta x) \times (y + \delta y) - x \times y|}{|x \times y|} \\
 &= \frac{|x\delta y + y\delta x + \delta x\delta y|}{|x \times y|} \\
 &\leq \frac{|x\delta y|}{|x \times y|} + \frac{|y\delta x|}{|x \times y|} + \frac{|\delta x\delta y|}{|x \times y|} \\
 &= \underbrace{\frac{|\delta y|}{|y|}}_{\text{erreur initiale sur } y} + \underbrace{\frac{|\delta x|}{|x|}}_{\text{erreur initiale sur } x} + \underbrace{\frac{|\delta x|}{|x|} \times \frac{|\delta y|}{|y|}}_{\text{terme négligeable}}
 \end{aligned}$$

Remarque 1.7 - sur l'expression

L'erreur relative sur le produit n'est que la somme des erreurs relatives initiales sur x et y , en plus d'un terme négligeable du second ordre, qu'on peut négliger dans un cadre simple. Dans la mesure où les ordres de grandeurs s'additionnent également dans un produit, on peut dire que le produit est une opération stable du point de vue des erreurs d'arrondi.

III Résolution de systèmes linéaires

Soit $A \in \mathcal{M}_n(\mathbb{R})$, $B \in \mathbb{R}^n$. En pratique, la solution numérique $x + \delta x$ de l'égalité vectorielle $AX = B$ vérifie :

$$A \times (x + \delta x) = b + \delta b$$

On a alors :

$$\delta b = A \times (x + \delta x) - b$$

Le vecteur x étant inconnu, on ne peut pas calculer directement δb . On cherche donc à majorer $\|\delta b\|$.

III.1 Rappels sur les normes usuelles sur $\mathbb{R}^n$

Définition 1.8 - norme infinie

Pour $X \in \mathbb{R}^n$, on définit la norme infinie par :

$$\|X\|_{\infty} = \max_{1 \leq i \leq n} |x_i|$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(X, inf)`.

Définition 1.9 - *norme de Mannhattan*

Pour $X \in \mathbb{R}^n$, on définit la norme de Manhattan par :

$$\|X\|_1 = \sum_{i=1}^n |x_i|$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(X, 1)`.

Définition 1.10 - *norme euclidienne*

Pour $X \in \mathbb{R}^n$, on définit la norme euclidienne par :

$$\|X\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(X)`.

Définition 1.11 - *norme subordonnée*

Pour $A \in \mathcal{M}_n(\mathbb{R})$, on définit la norme subordonnée associée à une norme vectorielle $\|\cdot\|$ par :

$$\|A\| = \sup_{X \in \mathbb{R}^n, X \neq 0} \frac{\|AX\|}{\|X\|}$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(A, ord)` où `ord` est l'ordre de la norme vectorielle associée (1, 2 ou `inf`).

Remarque 1.12 - *sur la norme euclidienne d'une matrice*

On a $\|A\|_2 = \sqrt{\rho(AA^T)}$ où $\rho(M)$ est la valeur propre de M de plus grande valeur absolue. Cette norme est difficile à calculer en pratique. On préfère donc utiliser les normes $\|\cdot\|_1$ ou $\|\cdot\|_\infty$ qui sont plus rapides à calculer.

III.2 Erreur d'approximation et méthode d'estimation de cette erreur

Si A est inversible, on a :

$$b = Ax \quad \Rightarrow \quad \|b\| = \|Ax\| \leq \|A\| \|x\|$$

De même :

$$\begin{aligned}
 A \times (x + \delta x) &= b + \delta b \\
 \Rightarrow A\delta x &= \delta b \\
 \Rightarrow \|\delta b\| &= \|A\delta x\| \leq \|A\| \|\delta x\| \\
 \Rightarrow \|b\| \|\delta b\| &\leq \|A\| \|x\| \|A^{-1}\| \|\delta x\| \\
 \Rightarrow \underbrace{\frac{\|\delta x\|}{\|x\|}}_{\text{erreur relative sur la solution}} &\leq \underbrace{\|A\| \|A^{-1}\|}_{\text{cond}(A)} \underbrace{\frac{\|\delta b\|}{\|b\|}}_{\text{erreur que l'on mesure}}
 \end{aligned}$$

L'erreur est amplifiée d'un facteur appelé *conditionnement* de la matrice A , noté :

$$\text{cond}(A) = \|A\| \|A^{-1}\|$$

En Python, on le calcule avec `cond(A, ord)` (cf. plus haut).

Définition 1.13 - mauvais conditionnement

On dit d'une matrice $A \in \mathcal{M}_n(\mathbb{R})$ qu'elle est *mal conditionnée* si son conditionnement est élevé :

- Si $\text{cond}(A) = 10^4$, alors A est mal conditionnée
- Si $\text{cond}(A) = 3$, alors A est bien conditionnée.

Moralement, pour résoudre $Ax = b$:

- On trouve $x + \delta x$
- On évalue le *résidu* $\|\delta b\| = \|b - A \times (x + \delta x)\|$

Remarque 1.14 - mauvais conditionnement

Les matrices obtenues en discrétisant des équations physiques sont généralement mal conditionnées.

IV Principes généraux pour réduire les erreurs d'arrondi

IV.1 Éviter de soustraire deux quantités absolument proches

Exemple 1.15 - soustraction au cosinus

Pour calculer $y = 1 - \cos(x)$, pour x proche de 0, on peut utiliser la formule :

$$\cos(2x) = 1 - 2 \sin^2(x)$$

pour obtenir :

$$y = 2 \sin^2(x/2)$$

Exemple 1.16

On s'intéresse à la recherche des racines de $P = aX^2 + bX + c$, de discriminant Δ vérifiant :

$$\Delta \gg 0$$

On a alors :

$$b^2 \gg 4ac$$

Ce discriminant est donc proche de b^2 . Ainsi, quand on calcule :

$$x_1 = \frac{-b - \Delta}{2a} \quad \text{et} \quad x_2 = \frac{-b + \Delta}{2a}$$

le calcul de x_2 est imprécis à cause du dénominateur. Mais via les formules de Viète, $x_2 = \frac{c}{ax_1}$.

IV.2 Sommer d'abord les termes les plus petits

Sommer un grand nombre avec un petit nombre entraîne la troncature des plus petites décimales du petit nombre.

Exemple 1.17 - calcul d'une somme

Pour calculer :

$$S = \sum_{i=1}^n \frac{1}{i}$$

bien qu'il soit naturel de sommer les termes dans l'ordre croissant des indices, l'ordre rétrograde est ici à privilégier. On calcule sur quatre décimales :

n	croissant	rétrograde	exact
100	5,187	5,187	5,1873775
1 000	7,449	7,485	7,485471
10 000	8,449	9,448	9,787606

V Calculs de complexité

Les additions, soustractions et multiplication ont aujourd'hui des coûts temporels semblables, mais la division est plus complexe. On cherche à s'en passer le plus possible.

Exemple 1.18 - complexité du produit d'une matrice avec un vecteur

Soit $A \in \mathcal{M}_{n,p}(\mathbb{R})$ et $x \in \mathbb{R}^n$. Notons $y = Ax$ et $y = (y_1, \dots, y_n)^\top$. On a :

$$\forall i \in \llbracket 1, n \rrbracket, y_i = \sum_{j=1}^p a_{i,j} x_j$$

. Le calcul de chaque y_i demande donc $2p$ opérations, ainsi le calcul de y demande $2np$ opérations.

Exemple 1.19 - résolution d'un système linéaire triangulaire

Calculons la complexité de la résolution de $Tx = b$. On prend T inférieure